Lab 3: Architectures and their costs

Fashion-MNIST | Five-epoch comparison | CPU, four threads

1. Data and training

The official 60,000 training images were split into 55,000 training and 5,000 validation examples with seed 42. The official 10,000 test images were used once per model after epoch five. Inputs use ToTensor and Normalize((0.2860,), (0.3530,)). All models use Adam, learning rate 0.001, batch size 128, and five epochs.

2. Architectures

MLP: Flatten 784 - Linear 256 - ReLU - Linear 128 - ReLU - Linear 10.
CNN: Conv(1,16,3,pad=1) - ReLU - MaxPool2 - Conv(16,32,3,pad=1) - ReLU - MaxPool2 - Flatten - Linear(1568,10).
Attention: 16 patches of 7x7; Linear(49,64); two Transformer encoder layers (width 64, four heads, feed-forward 128); mean pooling; Linear(64,10). Default dropout 0.1 and post-normalization; no added positional embeddings.

Conflicting requirement: the specified layers yield 235,146 / 20,490 / 70,794 parameters, rather than the stated 100k-150k target. The explicit architectures were preserved. Model size is a confounder; this is not an equal-size architecture comparison.

3. Measured results

Model	Parameters	Test accuracy	Batch 1 (ms)	Batch 128 (ms)
MLP	235,146	87.97%	0.1019	0.6108
CNN	20,490	89.64%	0.1438	12.1315
Attention	70,794	84.02%	0.3092	4.6009

Latency uses evaluation mode and torch.inference_mode, 20 warm-up runs per batch size, followed by the median of 100 runs for batch 1 and 20 runs for batch 128. Synthetic input tensors already reside on the device. Data loading and transfers are excluded. CUDA is synchronized if used. Fused attention fast paths were disabled to keep module profiling visible.

Environment: Python 3.11.8; PyTorch 2.14.1+cpu; Intel64 Family 6 Model 186 Stepping 3, GenuineIntel; four threads. Exact package versions are saved in results/environment.txt. Times are per batch; divide batch-128 latency by 128 for amortized per-image latency. Timing is hardware-dependent.

4. Hand calculations and profiling

Convention: a multiply and an add count as two FLOPs. Biases, activation functions, pooling, normalization, and softmax are excluded from the analytical matrix-operation baseline.

Linear: 2 x in x out. First MLP layer: 2 x 784 x 256 = **401,408** FLOPs. Weights plus biases: 784 x 256 + 256 = 200,960 parameters. Full MLP: 2 x (784 x 256 + 256 x 128 + 128 x 10) = **469,504**.

Convolution: 2 x K x K x C_in x C_out x H_out x W_out. First layer: 2 x 3 x 3 x 1 x 16 x 28 x 28 = **225,792**. Second layer = 1,806,336. Classifier = 31,360. Full CNN = **2,063,488** FLOPs.

Attention: sequence s=16, width d=64. Per encoder: QKV = 2 x 16 x 64 x 3 x 64 = 393,216; QK-transpose = 2 x 16^2 x 64 = 32,768; attention-times-V = 32,768; output projection = 131,072. Attention subtotal = 589,824. Feed-forward = 2 x 16 x (64 x 128 + 128 x 64) = 524,288. Two encoders = 2,228,224; patch projection = 100,352; classifier = 1,280. Full model = **2,329,856** FLOPs.

Model	Analytical FLOPs	Raw THOP MACs	Adjusted THOP x2
MLP	469,504	234,752	469,504
CNN	2,063,488	1,031,744	2,063,488
Attention	2,329,856	591,488	2,362,624

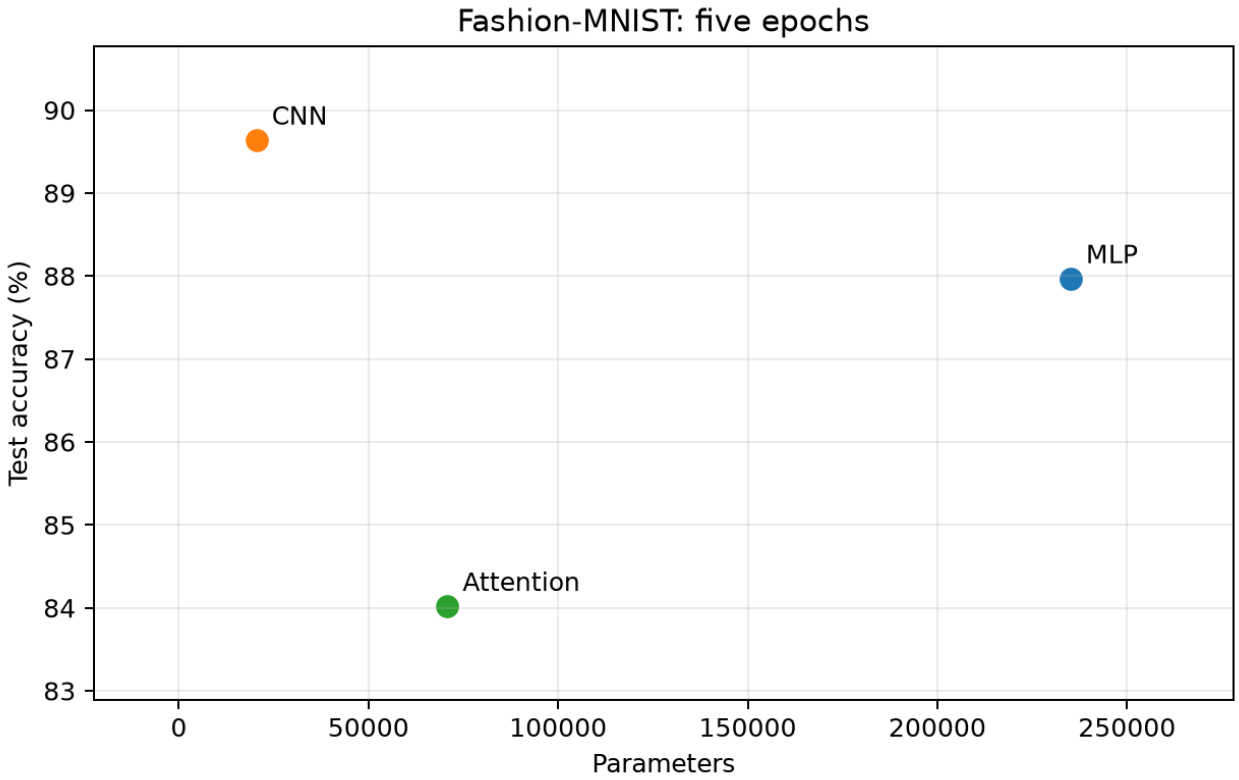
THOP reports mostly MACs rather than two-operation FLOPs. MLP and CNN match exactly after doubling. Raw THOP omits MultiheadAttention matrix operations. The custom hook restores 589,824 MACs across both encoders. The adjusted attention result exceeds the matrix-only baseline by 32,768 operations (1.41%), attributable to the four LayerNorm modules under THOP counting conventions. Doubling every tool-counted operation is an approximation. Parameter counts were also recorded with torchinfo.summary; summaries are in results/.

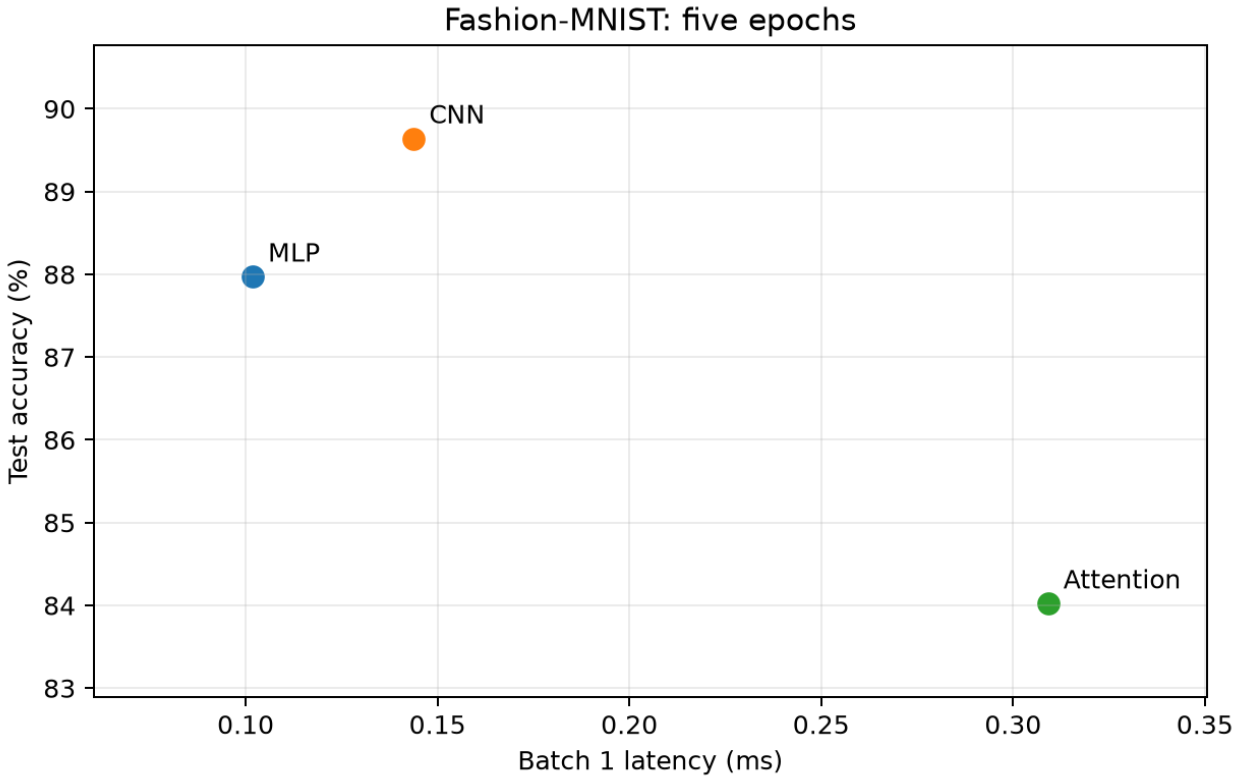
5. Access patterns and interpretation

CNN shares small kernels over local neighborhoods and reuses weights, but creates feature-map traffic. Pooling reduces later activation sizes. MLP streams a larger dense weight matrix; batching reuses weights across images and improves matrix multiplication efficiency. Attention adds token projections, score matrices, softmax, and normalization. Score storage grows quadratically with token count; at only 16 tokens, small-operation overhead can matter. These interpretations use the access-pattern topics named in Chapter 4.6-4.7; the chapter text was not supplied.

CNN provides the best observed accuracy/storage balance. MLP is fastest for both tested batch sizes. Attention offers no observed accuracy/latency advantage over MLP in this run. Five epochs and one seed support an initial comparison, not a general ranking of architectures. FLOPs alone omit memory traffic and execution overhead.

6. Accuracy and cost trade-offs





Conclusion: CNN achieves 89.64% test accuracy with 20,490 parameters. MLP reaches 87.97% and is fastest locally. Attention reaches 84.02%. Reproduce from lab03 with: python -m pip install -r requirements.txt; then python lab3.py --data data --threads 4.